



KAUST Academy Bioinformatics: Advanced Stage

Analysis based on Chromosome 5

Prepared by:

Wesam Bahitham

Mayar Alzaharani

Yara AlGhamdi

Omiaimah Algiraigri

Table of Contents

Introduction.....	2
Experimental Design	3
Methodology	4
Environment activation.....	6
Downloading Relevent Data	6
Quality Control	8
Calculating mapping rate.....	15
Differential Expression Analysis	16
Visualization	19
Conclusion	33

Introduction

TAR DNA-binding protein 43 (TDP-43) is a commonly expressed RNA-binding protein involved in multiple aspects of RNA metabolism, including splicing, stability, and transport. Loss of TDP-43 function has been strongly linked to neurodegenerative diseases such as amyotrophic lateral sclerosis (ALS) and frontotemporal dementia (FTD). Due to its broad regulatory role, TDP-43 knockout is expected to induce transcriptomic changes affecting numerous genes and biological pathways.

RNA sequencing provides a powerful approach to measure genome-wide gene expression and to identify transcriptional changes resulting from genetic perturbations such as TDP-43 loss. In this report, we performed a chromosome-specific RNA-seq analysis focusing on human chromosome 5. Chromosome-level analysis allows targeted investigation of localized transcriptional responses while reducing computational complexity. And a technical practice for the bioinformatics pipeline approach.

This report follows the steps practiced throughout the program, analysis will base as a building block to onward continuation. Ensembl release 115 and the GRCh38 genome assembly mark the versions used respective to consistency, we processed raw RNA-seq data from wild-type (WT) and TDP-43 knockout (KO) samples, performed quality control, aligned and quantified reads, and assessed mapping performance. Each lab builds on the previous one, forming a complete and coherent transcriptomic analysis pipeline aimed at identifying expression changes associated with TDP-43 loss on chromosome 5.

Project Objectives:

- Prepare chromosome-specific reference files for **chromosome 5**
- Perform quality control and preprocessing of RNA-seq data
- Quantify transcript abundance using a chromosome-specific transcriptome
- Compare mapping performance between WT and KO samples
- Establish a clean dataset suitable for downstream differential expression analysis
- Explicitly address task specific questions

Experimental Design

The dataset consists of paired-end RNA-seq reads generated from wild-type and TDP-43 knockout samples. Each condition contains three biological replicates (WT_1–WT_3 and KO_1–KO_3). All analyses were restricted to chromosome 5 to ensure consistency with the project design and to enable chromosome-focused interpretation of transcriptional effects.

Reference preparation → Data exploration → Quality control → Quantification → Evaluation of mapping performance

Dataset: TDP-43 Knockout Study (GSE136366)

Experiment and analysis of RNA-seq data comparing TDP-43 knockout (KO) vs wildtype rescue (WT) in human HeLa cells. TDP-43 is an RNA-binding protein implicated in ALS and frontotemporal dementia.

- **Samples:** 3 KO + 3 WT replicates (6 total)
- **Sequencing:** Paired-end Illumina HiSeq 2500, 70 bp reads
- **Workshop subset:** Chromosome 5

Methodology

1. Data Preparation

Initial lab set up included navigating through a linux system (i.e. shell scripting). Bioinformatics projects were created and organized large genomic datasets into and properly inspect them, this step is crucial in reproducibility integrity, as well as an efficient way to keep track. Due to the nature of science and research, the reference genome is consistently being updated, for this analysis version 115. After discussion and agreement, genome build used; GRCh38. These considerations follow best practices and ensure necessary considerations.

Initializing any analysis a dataset is required, samples were obtained and downloaded manually (6 paired samples, FASTQ FTP format). On that basis, the command line was used in accessing the human chromosome 5 reference from Ensemble (version 115). Accommodating the chromosome (5), the coordinates and metadata for genes, transcript and exons from a Gene Transfer Format (GTF) annotation file. FASTA and GTF files were decompressed. To wrap up and map the transcript ID and gene ID for respective files where mapped, crucial for pipeline analysis.

These steps ensure the foundation for the following analyses; Alignment, Quantification, and Differential Expression.

2. Quality control and read assessment

Following data preparation, quality control was performed on the raw sequencing reads. Read-level statistics such as total read counts, GC content, duplication levels, and adapter contamination were assessed. These metrics were summarized across samples to evaluate overall sequencing quality and identify potential technical issues before quantification.

3. Quantifying Transcripts

Transcript-level quantification was carried out using Salmon, which performs lightweight, alignment-free mapping of reads to the reference transcriptome derived from chromosome 5. Salmon outputs transcript abundance estimates in terms of TPM (Transcripts Per Million) and raw counts.

Mapping rates were extracted from Salmon log files to assess quantification efficiency.

4. Gene Level Aggregation

Transcript-level quantification results were prepared for downstream analysis by linking transcripts to their corresponding genes using the provided GTF annotation. Enabling gene-level expression summaries and forms the basis for exploratory data analysis and differential expression testing.

Constructing expression matrices to allow comparison (after transformation) between sample and conditions (WT, KO). Using this matrix further analysis was conducted i.e. differential expression analysis (DE), sample similarity evaluation, etc..

Environment activation

Creating subdirectories for each analysis step

```
# =====
# Project name and location
# =====
PROJ_BASE="$HOME/kaust_chr5_analysis"

# =====
# Creating project directory
# =====
mkdir -p "$PROJ_BASE"
cd "$PROJ_BASE"

mkdir -p subsampled_data
mkdir -p trimmed_data
mkdir -p references
mkdir -p qc_reports/fastqc_raw qc_reports/fastqc_trimmed qc_reports/fastp
mkdir -p alignment
mkdir -p salmon_quant
mkdir -p counts
mkdir -p results/tables results/figures results/enrichment
mkdir -p logs
mkdir -p raw_data
mkdir -p data
```

Downloading Relevant Data

Following code snippets include

- Samples were acquired from The European Nucleotide Archive
- Chromosome 5 from Ensembl fastQ FTP file
- GTF Gene Transcript File (gene annotation)

Sample directed to raw data file for reproducibility

```
# 1) Download RNA-seq data (chr5_data.zip) and unzip
# =====
ZENODO_URL="https://zenodo.org/records/18481303/files/chr5_data.zip?download=1"
ZIP_DESTINATION="data/chr5_data.zip"

if [ ! -f "$ZIP_DESTINATION" ]; then
    echo "Downloading data zip..."
    wget -O "$ZIP_DESTINATION" --content-disposition --trust-server-names "$ZENODO_URL"
    echo "Checking downloaded file type..."
    file "$ZIP_DESTINATION"
else
    echo "Data zip already exists: $ZIP_DESTINATION"
fi

echo "Unzipping data..."
unzip -o "$ZIP_DESTINATION" -d data/
echo

# Move FASTQ files to raw_data directory
if [ -d "data/chr5_data" ]; then
    echo "Moving FASTQ files to raw_data..."
    mv data/chr5_data/*.fastq.gz raw_data/ 2>/dev/null || true
fi
```

Downloading and setting up human chromosome 5 reference

```
# 2A) chr5 genome FASTA
CHR5_FASTA_URL="https://ftp.ensembl.org/pub/release-115/fasta/homo_sapiens/dna/Homo_sapiens.GRCh38.dna.chromosome.5.fa.gz"
CHR5_FASTA_COMPRESSED="references/Homo_sapiens.GRCh38.dna.chromosome.5.fa.gz"
CHR5_FASTA_UNCOMPRESSED="references/Homo_sapiens.GRCh38.dna.chromosome.5.fa"

if [ ! -f "$CHR5_FASTA_UNCOMPRESSED" ]; then
    echo "Downloading chr5 genome FASTA..."
    wget -O "$CHR5_FASTA_COMPRESSED" "$CHR5_FASTA_URL"
    gunzip -f "$CHR5_FASTA_COMPRESSED"
else
    echo "chr5 genome FASTA already exists"
fi
```

GTF reference file download

```
# 2B) GTF (full) + chr5-only GTF
ANNOTATION_URL="https://ftp.ensembl.org/pub/release-115/gtf/homo_sapiens/Homo_sapiens.GRCh38.115.gtf.gz"
ANNOTATION_COMPRESSED="references/Homo_sapiens.GRCh38.115.gtf.gz"
ANNOTATION_FULL="references/Homo_sapiens.GRCh38.115.gtf"
ANNOTATION_CHR5="references/Homo_sapiens.GRCh38.115.chr5.gtf"

if [ ! -f "$ANNOTATION_FULL" ]; then
    echo "Downloading GTF..."
    wget -O "$ANNOTATION_COMPRESSED" "$ANNOTATION_URL"
    gunzip -f "$ANNOTATION_COMPRESSED"
else
    echo "GTF already exists"
fi

if [ ! -f "$ANNOTATION_CHR5" ]; then
    echo "Creating chr5-only GTF..."
    awk 'BEGIN{OFS="\t"} $0 ~ /^#/ {print; next} $1=="5" {print}' "$ANNOTATION_FULL" > "$ANNOTATION_CHR5"
else
    echo "chr5 GTF already exists"
fi
```

This code downloads the human transcriptome (cDNA sequences) from Ensembl.

It checks if the file already exists locally - if not, it downloads the compressed version and extracts it. This transcriptome file contains all known mRNA sequences and is typically used for RNA-seq analysis, gene expression studies, or transcript-level alignment and quantification

```
# 2C) Transcriptome (cDNA) for Salmon
TRANSCRIPTOME_URL="https://ftp.ensembl.org/pub/release-115/fasta/homo_sapiens/cdna/Homo_sapiens.GRCh38.cdna.all.fa.gz"
TRANSCRIPTOME_COMPRESSED="references/Homo_sapiens.GRCh38.cdna.all.fa.gz"
TRANSCRIPTOME_UNCOMPRESSED="references/Homo_sapiens.GRCh38.cdna.all.fa"

if [ ! -f "$TRANSCRIPTOME_UNCOMPRESSED" ]; then
    echo "Downloading transcriptome (cDNA) for Salmon..."
    wget -O "$TRANSCRIPTOME_COMPRESSED" "$TRANSCRIPTOME_URL"
    gunzip -f "$TRANSCRIPTOME_COMPRESSED"
else
    echo "Transcriptome already exists"
fi
```


Following code performs quality control on raw sequencing data:

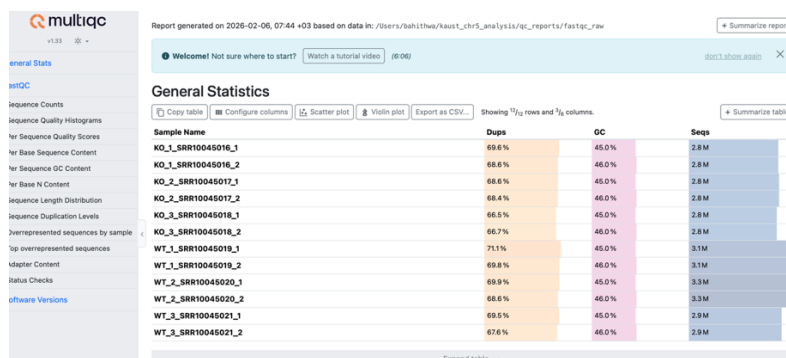
1. **Creates directory structure** for organizing QC reports
2. **Runs FastQC** on all raw FASTQ files (*.fastq.gz) to assess sequence quality, adapter content, and other metrics
3. **Runs MultiQC** to aggregate all individual FastQC reports into a single comprehensive HTML report for easy comparison across samples

This is a standard first step in any sequencing pipeline to identify quality issues before proceeding with downstream analysis.

```
cd "$(dirname "$0")/.."
mkdir -p qc_reports/fastqc_raw qc_reports/multiqc_raw
fastqc -o qc_reports/fastqc_raw raw_data/*.fastq.gz
multiqc -o qc_reports/multiqc_raw qc_reports/fastqc_raw
```

Quality Control

Aggregated files for easy comparison and analysis, using multiqc tool



fastp report

Summary

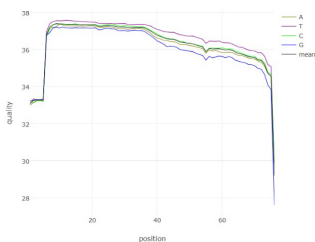
[illegible]

Adapters

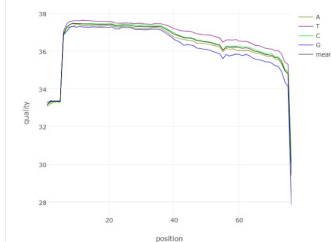
[illegible]

Filtering statistics

Before filtering: read1: quality



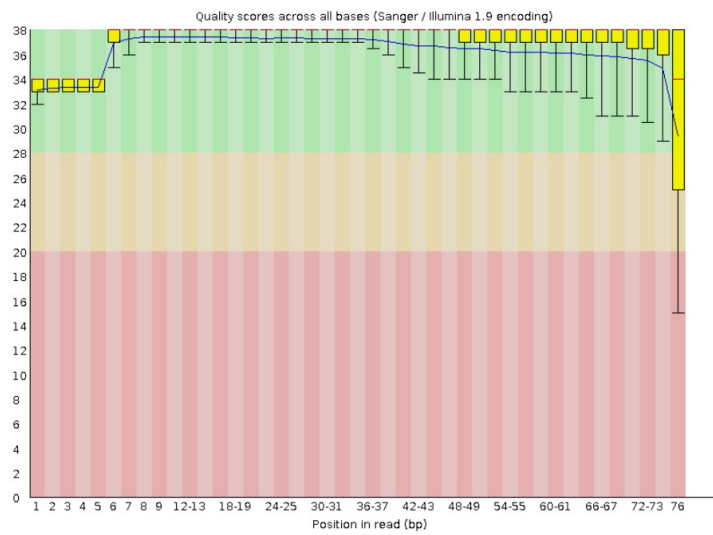
```
After filtering: read1: quality
```



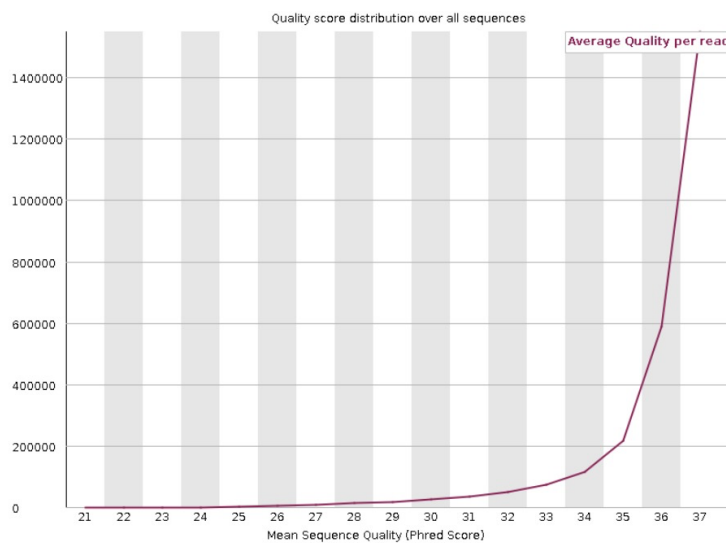
Basic Statistics

Measure	Value
Filename	KO_1_1.trim.fastq.gz
File type	Conventional base calls
Encoding	Sanger / Illumina 1.9
Total Sequences	2728443
Total Bases	207.3 Mbp
Sequences flagged as poor quality	0
Sequence length	36-76
%GC	45

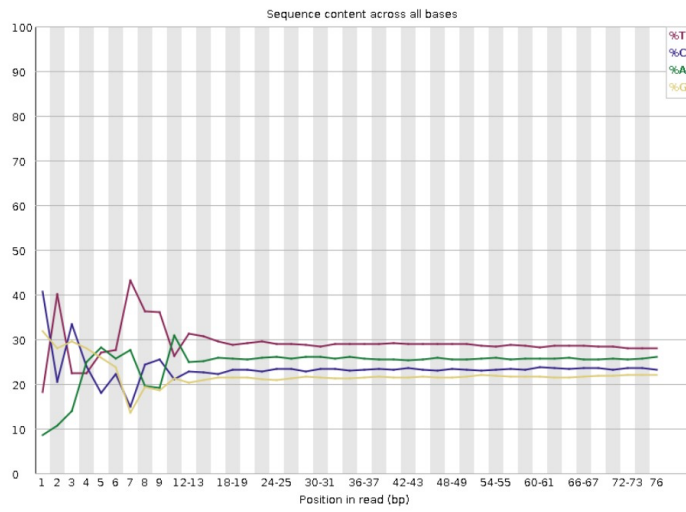
✓ Per base sequence quality



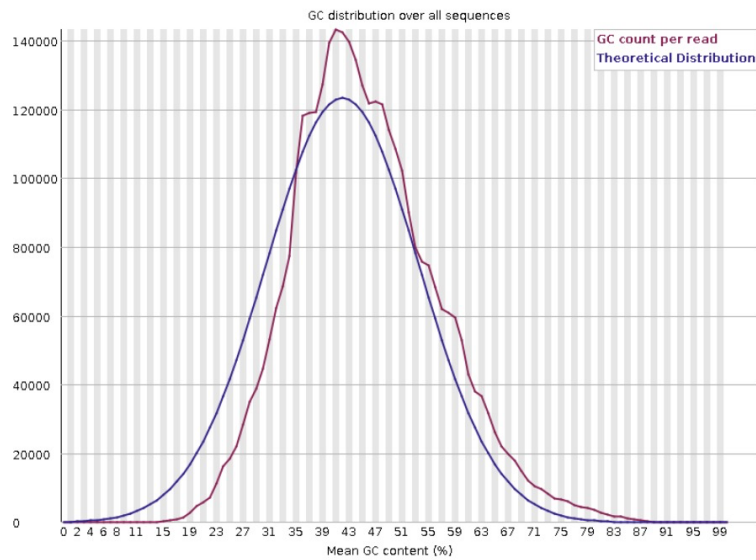
✓ Per sequence quality scores



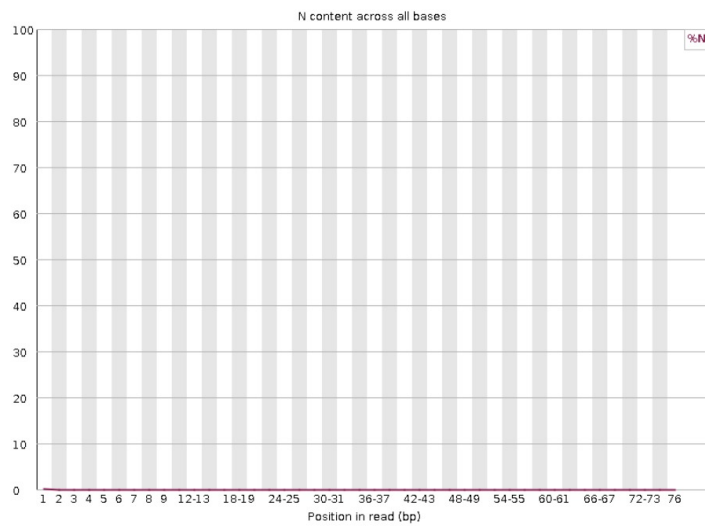
🚫 Per base sequence content



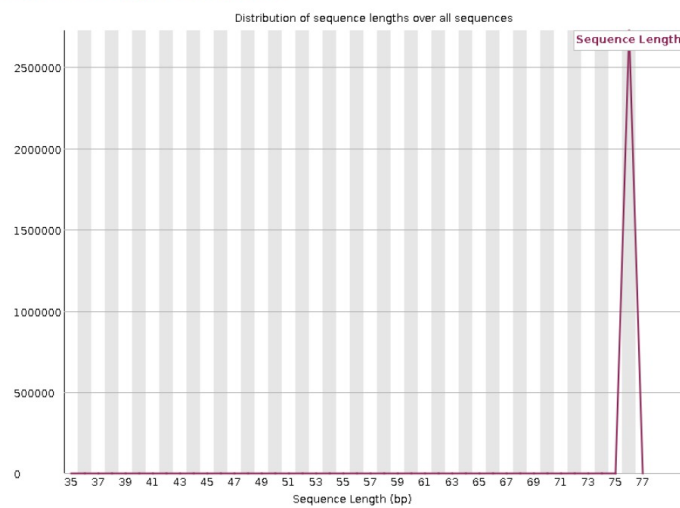
📊 Per sequence GC content



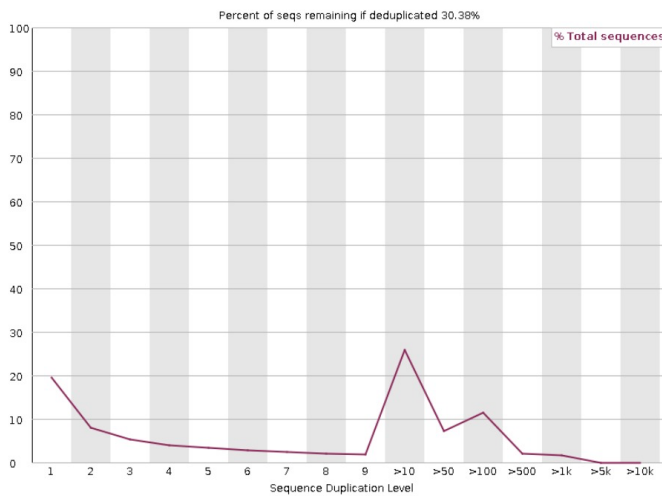
Per base N content



Sequence Length Distribution



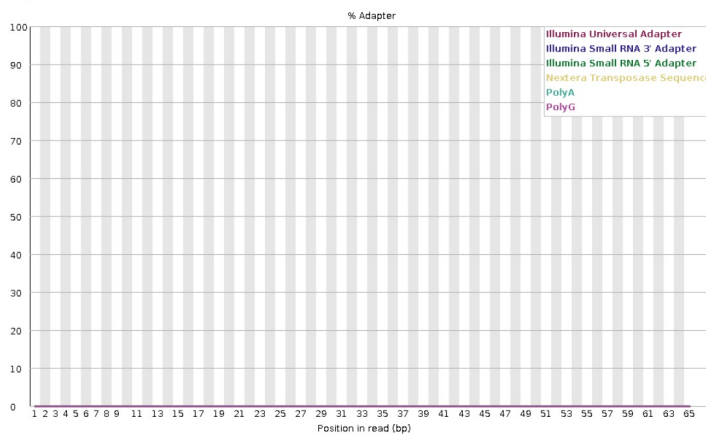
Sequence Duplication Levels



Overrepresented sequences

Sequence	Count	Percentage	Possible Source
CTCATTTTCATCATTATCCACCTTAAAGTGATAATCTTTGTCGGCCTTTA	3035	0.11123560213645659	No Hit

Adapter Content



Relevant statistics, analysis, and quality control summaries:

- High % PF (pass filter) across all samples: 95-98% - excellent
- Low adapter contamination: only 0.3% - great
- Consistent GC content: ~46% (expected for human samples)
- Good read depth: 2.6-3.2M reads per sample
- **Duplication rates are high:** 8-16% across samples, with KO and WT replicates showing 14-16% duplication. This is fairly common in RNA-seq but may indicate PCR over-amplification or low library complexity

For the purposes of this project, results are acceptable and good for practice (command shell prep, data downloading, pipeline analyses, etc.)

Data will be used for following downstream analysis:

General Statistics

Copy table | Configure columns | Scatter plot | Violin plot | Export as CSV... Showing 18/18 rows and 8/15 columns. + Summarize table

Sample Name	% Duplication	Reads After Filtering	GC content	% PF	% Adapter	Dups	GC	Seqs
KO_1_1						69.6 %	45.0 %	2.7 M
KO_1_2						69.4 %	46.0 %	2.7 M
KO_1_SRR10045016	15.0 %	5.5 M	46.1 %	97.7 %	0.3 %			
KO_2_1						68.8 %	45.0 %	2.8 M
KO_2_2						69.0 %	46.0 %	2.8 M
KO_2_SRR10045017	14.5 %	5.5 M	46.2 %	97.5 %	0.3 %			
KO_3_1						67.0 %	45.0 %	2.6 M
KO_3_2						67.4 %	46.0 %	2.6 M
KO_3_SRR10045018	8.3 %	5.2 M	46.0 %	95.3 %	0.3 %			
WT_1_1						71.2 %	45.0 %	3.0 M
WT_1_2						70.7 %	46.0 %	3.0 M
WT_1_SRR10045019	16.2 %	6.0 M	46.0 %	97.5 %	0.3 %			
WT_2_1						70.0 %	45.0 %	3.2 M
WT_2_2						69.3 %	46.0 %	3.2 M
WT_2_SRR10045020	15.1 %	6.4 M	45.8 %	97.6 %	0.3 %			
WT_3_1						69.8 %	45.0 %	2.8 M
WT_3_2						68.4 %	46.0 %	2.8 M
WT_3_SRR10045021	9.8 %	5.5 M	46.2 %	95.5 %	0.3 %			

Figure addresses “duplication level in the samples” (**Task 2, Question 3**)

Implicitly Salmon output TPM values are normalized for transcript length and sequencing depth. To explicitly answer (**Task 2, Question 7**) normalization will be systematically done from the DESeq2 R package.

RNA-seq quantification using Salmon

- **Sets up paths** for reference files and output directories
- **Builds Salmon index** from the transcriptome (if not already built) - this creates a searchable database of transcripts
- **Loops through all samples** (WT and KO replicates) and quantifies transcript abundance using quasi-mapping
- **Uses 4 threads** for faster processing

```
cd "$(dirname "$0")/.."

REFERENCE_PATH="references"
TRIMMED_PATH="trimmed_data"
SALMON_INDEX_PATH="${REFERENCE_PATH}/salmon_index"
QUANTIFICATION_OUTPUT="salmon_quant"
TRANSCRIPTOME_FILE="${REFERENCE_PATH}/Homo_sapiens.GRCh38.cdna.all.fa"
NUM_THREADS="${THREADS:-8}"

mkdir -p "${SALMON_INDEX_PATH}" "${QUANTIFICATION_OUTPUT}"

if [ ! -f "${TRANSCRIPTOME_FILE}" ]; then
    echo "ERROR: Missing transcriptome FASTA: ${TRANSCRIPTOME_FILE}"
    exit 1
fi

if [ ! -f "${SALMON_INDEX_PATH}/versionInfo.json" ]; then
    echo "Building Salmon index..."
    salmon index -t "${TRANSCRIPTOME_FILE}" -i "${SALMON_INDEX_PATH}" -p "${NUM_THREADS}"
else
    echo "Salmon index exists: ${SALMON_INDEX_PATH}"
fi

SAMPLE_NAMES=(WT_1 WT_2 WT_3 KO_1 KO_2 KO_3)

for sample_name in "${SAMPLE_NAMES[@]"; do
    FORWARD_READ="${TRIMMED_PATH}/${sample_name}_1.trim.fastq.gz"
    REVERSE_READ="${TRIMMED_PATH}/${sample_name}_2.trim.fastq.gz"
    SAMPLE_OUTPUT_DIR="${QUANTIFICATION_OUTPUT}/${sample_name}"

    if [ ! -f "${FORWARD_READ}" ] || [ ! -f "${REVERSE_READ}" ]; then
        echo "ERROR: Missing trimmed reads for ${sample_name}:"
        echo "  ${FORWARD_READ}"
        echo "  ${REVERSE_READ}"
        exit 1
    fi

    echo "Quantifying ${sample_name} ..."
    salmon quant \
        -i "${SALMON_INDEX_PATH}" \
        -l A \
        -1 "${FORWARD_READ}" \
        -2 "${REVERSE_READ}" \
        -p "${NUM_THREADS}" \
        -o "${SAMPLE_OUTPUT_DIR}"
done

echo "Done. Results in: ${QUANTIFICATION_OUTPUT}"
```

Calculating mapping rate

Addressing “Minimum and Maximum A alignment rate across sample” (Task 2, Question 2)

Mapping Rate Summary:

- WT_1: 95.38%
- WT_2: 93.99% *minimum*
- WT_3: 94.89%
- KO_1: 95.34% *maximum*
- KO_2: 94.78%
- KO_3: 94.73%

These are **excellent mapping rates** across all samples. Typically, mapping rates above 70-80% are considered acceptable for RNA-seq, rates in the mid-90s range, which indicates:

1. **High-quality reads** - sequencing data is clean with minimal adapter contamination or low-quality bases
2. **Appropriate reference genome** - The salmon index is well-matched to samples

3. **Low contamination** - Minimal presence of non-target sequences (e.g., rRNA, microbial DNA)
4. **Good library prep** - The RNA extraction and library preparation were successful

Consistency: The rates are very consistent between replicates (both WT and KO groups show <2% variation).

The data quality is more than sufficient for robust identification of genes affected by knockout. Analysis will continue:

```
(genomics) bahithwa@k1-25367 ~/kaust_chr5_analysis/salmon_quant % for sample in WT_1 WT_2 WT_3 KO_1 KO_2 KO_3; do echo "=== $sample ==="; g]
rep "Mapping rate" ${sample}/logs/salmon_quant.log; done
=== WT_1 ===
[2026-02-06 07:53:43.118] [jointLog] [info] Mapping rate = 95.3081%
=== WT_2 ===
[2026-02-06 07:54:02.554] [jointLog] [info] Mapping rate = 93.9855%
=== WT_3 ===
[2026-02-06 07:54:22.172] [jointLog] [info] Mapping rate = 94.8899%
=== KO_1 ===
[2026-02-06 07:54:37.402] [jointLog] [info] Mapping rate = 95.3412%
=== KO_2 ===
[2026-02-06 07:54:53.471] [jointLog] [info] Mapping rate = 94.7798%
=== KO_3 ===
[2026-02-06 07:55:10.715] [jointLog] [info] Mapping rate = 94.7298%
```

Differential Expression Analysis

```
(genomics) abdullah@Abdullah:~/kaust_chr5_analysis$ head -20 counts/gene_cou
nts.tsv | column -t
gene_id KO_1 KO_2 KO_3 WT_1 WT_2 WT_3
ENSG00000006837 61 84 69 47 72 50
ENSG00000011083 0 0 0 0 0 0
ENSG00000013561 2302 2332 2431 2320 2257 2239
ENSG00000015479 26542 25570 25148 30569 32743 27018
ENSG00000016082 83 67 64 55 56 53
ENSG00000019582 3 3 0 3 18 3
ENSG00000022355 5 4 6 69 59 58
ENSG00000027847 637 697 579 576 687 562
ENSG00000028310 3816 3988 3936 6125 7705 5979
ENSG00000031003 3268 3109 3073 3880 4137 3474
ENSG00000035499 1182 1247 1105 1148 1305 1134
ENSG00000037241 2120 2200 1867 2419 2633 2140
ENSG00000037280 0 1 1 3 3 1
ENSG00000037474 28116 27990 25228 34077 35464 30162
ENSG00000037749 2897 3037 3188 3281 3493 3282
ENSG00000038274 4011 3850 3768 4775 4902 4467
ENSG00000038382 7169 8193 6724 8029 9270 7165
ENSG00000038427 0 0 3 0 0 0
ENSG00000039123 7588 7521 7341 8674 8815 7420

(genomics) abdullah@Abdullah:~/kaust_chr5_analysis$ wc -l counts/gene_counts
.tsv
1574 counts/gene_counts.tsv

(genomics) abdullah@Abdullah:~/kaust_chr5_analysis$ cat counts/sample_info.t
sv
sample_id condition
KO_1 KO
KO_2 KO
KO_3 KO
WT_1 WT
WT_2 WT
WT_3 WT
```

```

1. # Summary of differential expression results
2. df = deseq_results.dropna(subset=['padj'])
3.
4. # Count genes at different thresholds
5. thresholds = [
6.     ('padj < 0.1', (df['padj'] < 0.1).sum()),
7.     ('padj < 0.05', (df['padj'] < 0.05).sum()),
8.     ('padj < 0.01', (df['padj'] < 0.01).sum()),
9.     ('padj < 0.05 & |LFC| > 1', ((df['padj'] < 0.05) & (abs(df['log2FoldChange']) > 1)).sum()),
10.    ('padj < 0.05 & |LFC| > 2', ((df['padj'] < 0.05) & (abs(df['log2FoldChange']) > 2)).sum()),
11. ]
12.
13. print("=" * 50)
14. print("Summary of Differential Expression Analysis")
15. print("=" * 50)
16. print(f"\nTotal genes tested: {len(df)}")
17. print("\nGenes at different significance thresholds:")
18. for desc, count in thresholds:
19.     print(f"    {desc}: {count}")
20.
21. # Up vs Down regulated
22. sig = df[(df['padj'] < 0.05) & (abs(df['log2FoldChange']) > 1)]
23. up = (sig['log2FoldChange'] > 0).sum()
24. down = (sig['log2FoldChange'] < 0).sum()
25. print(f"\nAt padj < 0.05 and |LFC| > 1:")
26. print(f"    Upregulated in KO: {up}")
27. print(f"    Downregulated in KO: {down}")

```

Output:

Summary of Differential Expression Analysis

Total genes tested: 566

Genes at different significance thresholds:

padj < 0.1: 269

padj < 0.05: 227

padj < 0.01: 165

padj < 0.05 & |LFC| > 1: 29

padj < 0.05 & |LFC| > 2: 10

At padj < 0.05 and |LFC| > 1:

Upregulated in KO: 26

Downregulated in KO: 3

```
1. txi <- readRDS("~/kaust_chr5_analysis/counts/tximport.rds")
2. sample_info <- read.table("~/kaust_chr5_analysis/counts/sample_info.tsv", header = TRUE, sep = "\t")
3.
4. # Set factor levels (WT as reference)
5. sample_info$condition <- factor(sample_info$condition, levels = c("WT", "KO"))
6.
7. cat(paste0("Samples: ", nrow(sample_info), "\n"))
8. cat(paste0("Genes: ", nrow(txi$counts), "\n"))
9. cat(paste0("Conditions: ", paste(levels(sample_info$condition), collapse = " vs "), "\n"))
10.
```

Output:

Samples: 6

Genes: 1573

Conditions: WT vs KO

```
1. dds <- DESeqDataSetFromTximport(txi, colData = sample_info, design = ~ condition)
2.
3. # Pre-filtering: Remove genes with very low counts
4. keep <- rowSums(counts(dds) >= 10) >= 3
5. dds <- dds[keep, ]
6. cat(paste0("Genes after filtering (>=10 counts in >=3 samples): ", nrow(dds), "\n"))
7.
```

Output:

Genes after filtering (>=10 counts in >=3 samples): 566

```
1. dds <- DESeq(dds)
2. res <- results(dds, contrast = c("condition", "KO", "WT"))
3. res <- res[order(res$padj), ]
4.
5. summary(res)
6.
7. # Significant: padj < 0.05 and |log2FC| > 1
8. sig_genes <- subset(res, padj < 0.05 & abs(log2FoldChange) > 1)
9. sig_genes <- sig_genes[order(sig_genes$padj), ]
10.
11. cat(paste0("Total significant genes: ", nrow(sig_genes), "\n"))
12. cat(paste0("Upregulated in KO: ", sum(sig_genes$log2FoldChange > 0, na.rm = TRUE), "\n"))
13. cat(paste0("Downregulated in KO: ", sum(sig_genes$log2FoldChange < 0, na.rm = TRUE), "\n"))
```

Output:

out of 566 with nonzero total read count

adjusted p-value < 0.1

LFC > 0 (up) : 129, 23%

LFC < 0 (down) : 140, 25%

outliers [1] : 0, 0%

low counts [2] : 0, 0%

(mean count < 10)

[1] see 'cooksCutoff' argument of ?results

[2] see 'independentFiltering' argument of ?results

Total significant genes: 29

Upregulated in KO: 26

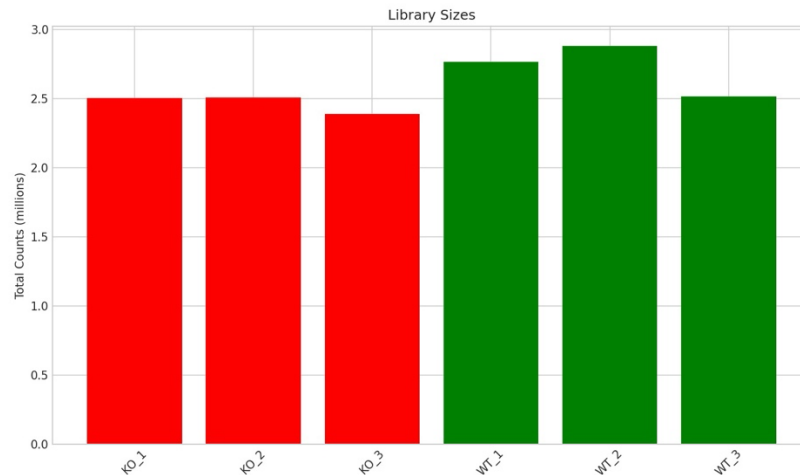
Downregulated in KO: 3

```
1. counts = pd.read_csv(os.path.expanduser("~/kaust_chr5_analysis/counts/gene_counts.tsv"), sep="\t", index_col=0)
2.
3. # Display summary
4. print(f"Gene counts matrix: {counts.shape[0]} genes x {counts.shape[1]} samples")
5. print(f"Samples: {list(counts.columns)}")
6. counts.head()
7.
```

gene_id						
ENSG00000006837	61	84	69	47	72	50
ENSG00000011083	0	0	0	0	0	0
ENSG00000013561	2302	2332	2431	2320	2257	2239
ENSG00000015479	26542	25570	25148	30569	32743	27018
ENSG00000016082	83	67	64	55	56	53

Visualization

```
1. fig, ax = plt.subplots(figsize=(10, 6))
2. library_sizes = counts.sum()
3. colors = ['red' if 'KO' in s else 'green' for s in library_sizes.index]
4. ax.bar(library_sizes.index, library_sizes.values / 1e6, color=colors)
5. ax.set_ylabel("Total Counts (millions)")
6. ax.set_title("Library Sizes")
7. plt.xticks(rotation=45)
8. plt.tight_layout()
9. plt.savefig(f"{output_dir}/library_sizes.png", dpi=150)
10. plt.show()
11. print(f"Saved to {output_dir}/library_sizes.png")
12.
```

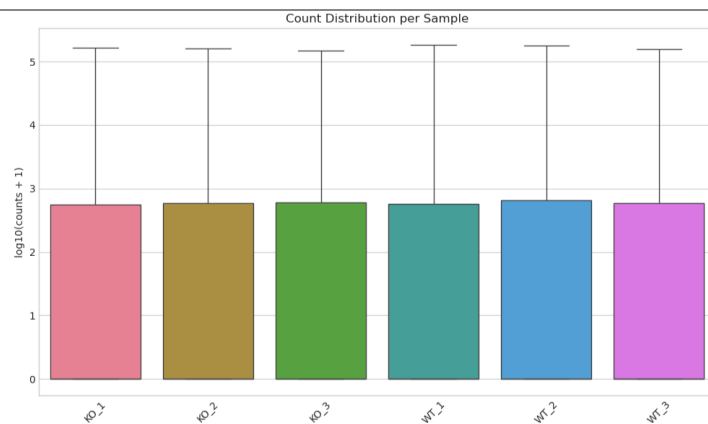


Library sizes are consistent across all six samples (~2.4–2.9M reads), with no major outliers, indicating comparable sequencing depth between KO and WT groups.

```

1. fig, ax = plt.subplots(figsize=(10, 6))
2. log_counts = np.log10(counts + 1)
3.
4. # Use seaborn to plot
5. sns.boxplot(data=log_counts, ax=ax)
6.
7. ax.set_ylabel("log10(counts + 1)")
8. ax.set_title("Count Distribution per Sample")
9. plt.xticks(rotation=45)
10. plt.tight_layout()
11. plt.savefig(f"{output_dir}/count_boxplot_distribution.png", dpi=150)
12. plt.show()
13. print(f"Saved to {output_dir}/count_boxplot_distribution.png")
14.

```



Log-transformed count distributions are uniform across all samples, confirming consistent data quality and no sample-level technical bias requiring correction.

```

1. fig, ax = plt.subplots(figsize=(10, 6))
2. log_counts = np.log10(counts + 1)
3.
4. # Use seaborn to plot
5. sns.violinplot(data=log_counts, ax=ax)
6.
7. ax.set_ylabel("log10(counts + 1)")
8. ax.set_title("Count Distribution per Sample")
9. plt.xticks(rotation=45)
10. plt.tight_layout()
11. plt.savefig(f"{output_dir}/count_boxplot_distribution.png", dpi=150)
12. plt.show()
13. print(f"Saved to {output_dir}/count_boxplot_distribution.png")
14.
15.

```

```

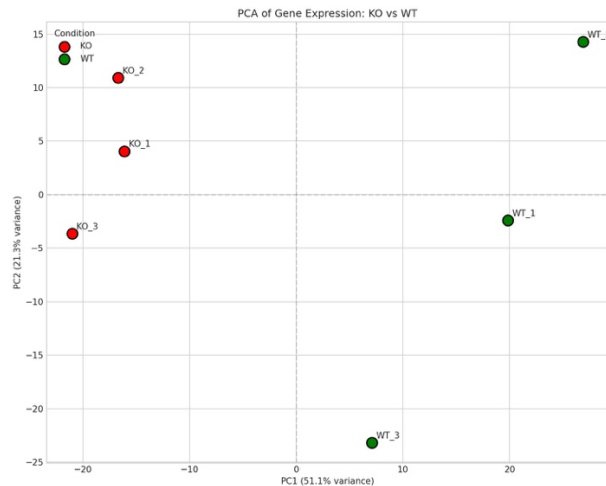
1. counts_filtered = counts[counts.max(axis=1) >= 3]
2. print(f"Genes after filtering: {len(counts_filtered)}")
3.
4. # Log transform the data
5. log_counts = np.log2(counts_filtered + 1)
6.
7. # Transpose so samples are rows
8. data_for_pca = log_counts.T
9.
10. # Standardize
11. scaler = StandardScaler()
12. data_scaled = scaler.fit_transform(data_for_pca)
13.
14. # Perform PCA
15. pca = PCA(n_components=2)
16. pca_result = pca.fit_transform(data_scaled)
17.
18. # Create dataframe with results
19. pca_df = pd.DataFrame({
20.     'PC1': pca_result[:, 0],
21.     'PC2': pca_result[:, 1],
22.     'Sample': data_for_pca.index,
23.     'Condition': ['KO' if 'KO' in s else 'WT' for s in data_for_pca.index]
24. })
25.
26. # Plot
27. fig, ax = plt.subplots(figsize=(10, 8))
28. colors = {'KO': 'red', 'WT': 'green'}
29.
30. for condition in ['KO', 'WT']:
31.     mask = pca_df['Condition'] == condition
32.     ax.scatter(pca_df.loc[mask, 'PC1'], pca_df.loc[mask, 'PC2'],
33.               c=colors[condition], label=condition, s=150,
34.               edgecolors='black', linewidth=1.5)
35.
36. # Add sample labels
37. for idx, row in pca_df.iterrows():
38.     ax.annotate(row['Sample'], (row['PC1'], row['PC2']),
39.               xytext=(5, 5), textcoords='offset points', fontsize=10)
40.
41. ax.set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
42. ax.set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")

```

```

43. ax.set_title("PCA of Gene Expression: KO vs WT")
44. ax.legend(title="Condition", loc='best')
45. ax.axhline(y=0, color='gray', linestyle='--', alpha=0.3)
46. ax.axvline(x=0, color='gray', linestyle='--', alpha=0.3)
47.
48. plt.tight_layout()
49. plt.savefig(f'{output_dir}/pca_ko_vs_wt.png', dpi=150)
50. plt.show()
51. print(f'Saved to {output_dir}/pca_ko_vs_wt.png')
52.

```



PCA shows clear separation between KO and WT along PC1 (51.1% variance), confirming the knockout drives the dominant transcriptional difference, though WT_3 is an outlier on PC2.

Genes after filtering: 700

```

1. # Load DESeq2 results
2. deseq_results = pd.read_csv(
3.     os.path.expanduser("~/kaust_chr5_analysis/results/tables/deseq2_all_results.csv")
4. )
5.
6. # Load normalized counts
7. norm_counts = pd.read_csv(
8.     os.path.expanduser("~/kaust_chr5_analysis/results/tables/normalized_counts.csv"),
9.     index_col='gene_id'
10. )
11.
12. print(f'Total genes in results: {len(deseq_results)}')
13. print(f"\nDESeq2 results preview:")
14. deseq_results.head()
15.

```

	gene_id	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj
0	ENSG00000113140	2003.333787	3.055275	0.089221	34.243905	5.374785e-257	3.042128e-254
1	ENSG00000213949	1063.766503	1.386158	0.114097	12.148932	5.811961e-34	1.644785e-31
2	ENSG00000174705	6429.386353	0.483132	0.044219	10.926000	8.658095e-28	1.633494e-25
3	ENSG00000134352	11527.217995	0.504619	0.048447	10.415815	2.099913e-25	2.971377e-23
4	ENSG00000120129	1837.321113	0.739232	0.072677	10.171441	2.659440e-24	3.010486e-22

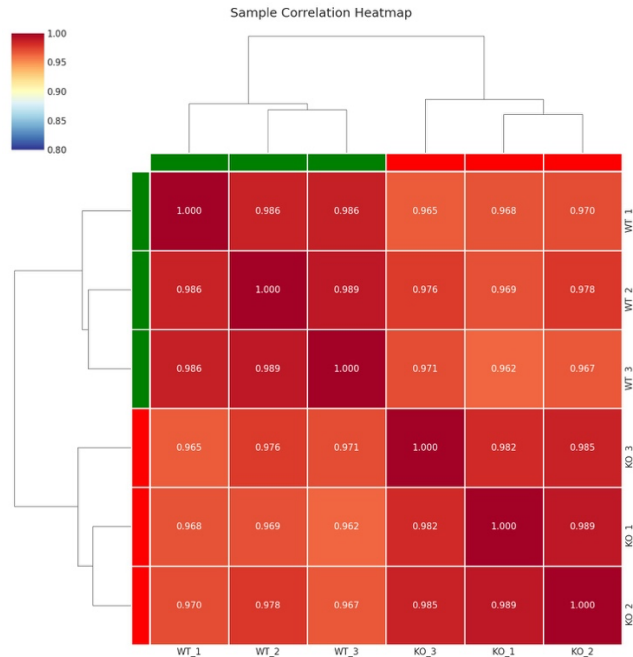
Total genes in results: 566

DESeq2 results preview:

```

1. # Sample-to-sample correlation heatmap
2. log_counts = np.log2(norm_counts + 1)
3.
4. # Calculate correlation matrix
5. corr_matrix = log_counts.corr()
6.
7. # Create annotation for conditions
8. sample_conditions = ['KO' if 'KO' in s else 'WT' for s in corr_matrix.columns]
9. condition_colors = {'KO': 'red', 'WT': 'green'}
10. row_colors = [condition_colors[c] for c in sample_conditions]
11.
12. # Plot clustered heatmap
13. g = sns.clustermap(corr_matrix,
14.                    method='average',
15.                    cmap='RdYlBu_r',
16.                    vmin=0.8, vmax=1.0,
17.                    figsize=(10, 10),
18.                    row_colors=row_colors,
19.                    col_colors=row_colors,
20.                    annot=True,
21.                    fmt='.3f',
22.                    linewidths=0.5)
23.
24. g.fig.suptitle('Sample Correlation Heatmap', y=1.02, fontsize=14)
25.
26. plt.savefig(f'{output_dir}/sample_correlation_heatmap.png', dpi=150, bbox_inches='tight')
27. plt.show()
28. print(f'Saved to {output_dir}/sample_correlation_heatmap.png')

```

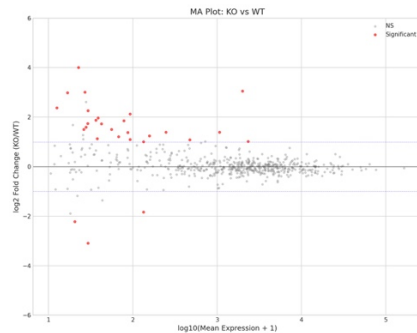



High pairwise correlations (>0.96) across all samples with hierarchical clustering correctly grouping KO and WT separately, confirming strong within-group similarity and between-group distinction.

```

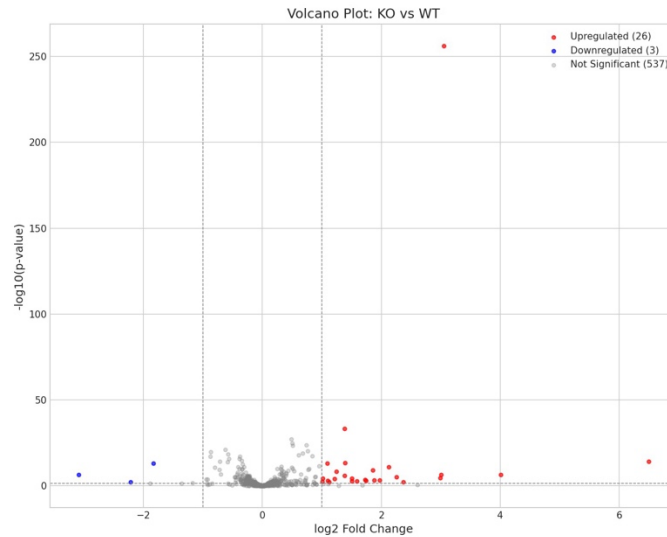
1. # MA Plot: log2 Fold Change vs Mean Expression
2. fig, ax = plt.subplots(figsize=(10, 8))
3.
4. df = deseq_results.copy()
5. df['significant'] = (df['padj'] < 0.05) & (abs(df['log2FoldChange']) > 1)
6.
7. # Plot non-significant genes
8. ns = df[~df['significant']]
9. ax.scatter(np.log10(ns['baseMean'] + 1), ns['log2FoldChange'],
10.            c='gray', alpha=0.3, s=10, label='NS')
11.
12. # Plot significant genes
13. sig = df[df['significant']]
14. ax.scatter(np.log10(sig['baseMean'] + 1), sig['log2FoldChange'],
15.            c='red', alpha=0.6, s=15, label='Significant')
16.
17. # Add reference lines
18. ax.axhline(y=0, color='black', linestyle='-', linewidth=0.5)
19. ax.axhline(y=1, color='blue', linestyle='--', linewidth=0.5, alpha=0.5)
20. ax.axhline(y=-1, color='blue', linestyle='--', linewidth=0.5, alpha=0.5)
21.
22. ax.set_xlabel('log10(Mean Expression + 1)', fontsize=12)
23. ax.set_ylabel('log2 Fold Change (KO/WT)', fontsize=12)
24. ax.set_title('MA Plot: KO vs WT', fontsize=14)
25. ax.legend(loc='upper right')
26. ax.set_ylim(-6, 6)
27.
28. plt.tight_layout()
29. plt.savefig(f'{output_dir}/MA_plot.png', dpi=150, bbox_inches='tight')
30. plt.show()
31. print(f'Saved to {output_dir}/MA_plot.png')

```



MA plot shows significant DE genes (red) are predominantly low-to-moderately expressed with a bias toward upregulation in KO, while highly expressed genes remain largely unchanged.

```
1. # Volcano Plot: -log10(p-value) vs log2 Fold Change
2. fig, ax = plt.subplots(figsize=(10, 8))
3.
4. df = deseq_results.copy()
5. df = df.dropna(subset=['pvalue', 'log2FoldChange'])
6.
7. # Calculate -log10(pvalue)
8. df['neg_log10_pval'] = -np.log10(df['pvalue'])
9.
10. # Classify genes
11. conditions = [
12.     (df['padj'] < 0.05) & (df['log2FoldChange'] > 1), # Up
13.     (df['padj'] < 0.05) & (df['log2FoldChange'] < -1), # Down
14. ]
15. choices = ['Upregulated', 'Downregulated']
16. df['regulation'] = np.select(conditions, choices, default='Not Significant')
17.
18. # Define colors
19. colors = {'Upregulated': 'red', 'Downregulated': 'blue', 'Not Significant': 'gray'}
20.
21. # Plot each category
22. for reg, color in colors.items():
23.     subset = df[df['regulation'] == reg]
24.     alpha = 0.7 if reg != 'Not Significant' else 0.3
25.     ax.scatter(subset['log2FoldChange'], subset['neg_log10_pval'],
26.               c=color, alpha=alpha, s=15, label=f'{reg} ({len(subset)})')
27.
28. # Add threshold lines
29. ax.axvline(x=1, color='gray', linestyle='--', linewidth=0.8)
30. ax.axvline(x=-1, color='gray', linestyle='--', linewidth=0.8)
31. ax.axhline(y=-np.log10(0.05), color='gray', linestyle='--', linewidth=0.8)
32.
33. ax.set_xlabel('log2 Fold Change', fontsize=12)
34. ax.set_ylabel('-log10(p-value)', fontsize=12)
35. ax.set_title('Volcano Plot: KO vs WT', fontsize=14)
36. ax.legend(loc='upper right')
37.
38. plt.tight_layout()
39. plt.savefig(f'{output_dir}/volcano_plot.png', dpi=150, bbox_inches='tight')
40. plt.show()
41. print(f'Saved to {output_dir}/volcano_plot.png')
```



Volcano plot reveals a strong directional bias with 26 upregulated vs only 3 downregulated genes in KO, with one gene showing extreme significance ($-\log_{10} p\text{-value} > 250$) suggesting a highly robust knockout effect.

```

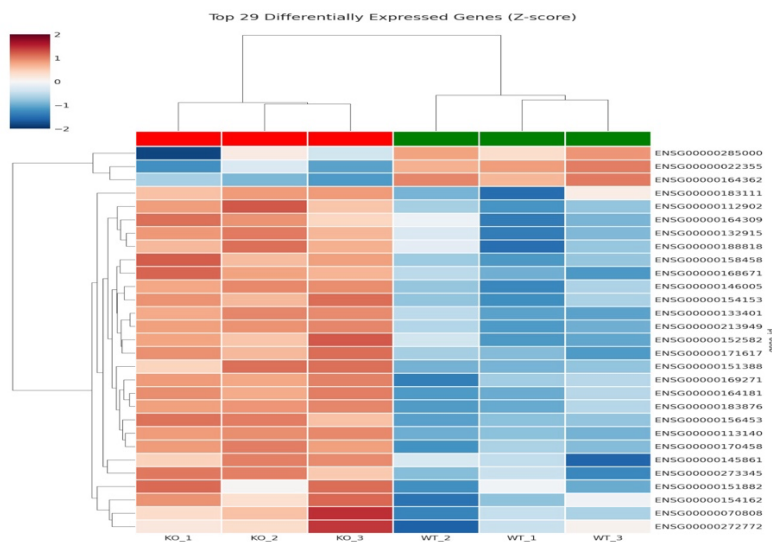
1. # Heatmap of top DE genes
2. sig_genes = deseq_results[
3.     (deseq_results['padj'] < 0.05) &
4.     (abs(deseq_results['log2FoldChange']) > 1)
5. ].sort_values('padj')
6.
7. print(f"Total significant genes: {len(sig_genes)}")
8.
9. # Select top 30 genes
10. n_top = min(30, len(sig_genes))
11. top_genes = sig_genes.head(n_top)['gene_id'].tolist()
12.
13. if len(top_genes) > 0:
14.     # Extract counts for top genes
15.     top_counts = norm_counts.loc[norm_counts.index.isin(top_genes)]
16.
17.     # Log transform and Z-score normalize
18.     log_top = np.log2(top_counts + 1)
19.     z_scores = (log_top.T - log_top.mean(axis=1)) / log_top.std(axis=1)
20.     z_scores = z_scores.T
21.
22.     # Create annotation
23.     sample_conditions = ['KO' if 'KO' in s else 'WT' for s in z_scores.columns]
24.     col_colors = ['red' if c == 'KO' else 'green' for c in sample_conditions]
25.
26.     # Plot
27.     g = sns.clustermap(z_scores,
28.                        method='average',
29.                        cmap='RdBu_r',
30.                        center=0,
31.                        vmin=-2, vmax=2,
32.                        figsize=(10, 12),
33.                        col_colors=col_colors,
34.                        yticklabels=True,
35.                        xticklabels=True,
36.                        linewidths=0.5)
37.
38.     g.fig.suptitle(f'Top {n_top} Differentially Expressed Genes (Z-score)', y=1.02, fontsize=14)

```

```

39.
40. plt.savefig(f'{output_dir}/top_genes_heatmap.png', dpi=150, bbox_inches='tight')
41. plt.show()
42. print(f'Saved to {output_dir}/top_genes_heatmap.png')
43. else:
44.     print("No significant genes found with the current thresholds.")
45.

```



Heatmap of the top 29 DE genes shows a clear expression dichotomy between KO and WT, with the majority upregulated in KO (red) and consistently low in WT (blue), reinforcing the directional bias seen in the volcano plot.

```

1. # Load all DE results
2. de_results = pd.read_csv('results/tables/deseq2_all_results.csv')
3.
4. print(f'Total genes in DE results: {len(de_results)}')
5. print(f'Columns: {list(de_results.columns)}')
6. de_results.head()
7.

```

Total genes in DE results: 566

Columns: ['gene_id', 'baseMean', 'log2FoldChange', 'lfcSE', 'stat', 'pvalue', 'padj']

	gene_id	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj
0	ENSG00000113140	2003.333787	3.055275	0.089221	34.243905	5.374785e-257	3.042128e-254
1	ENSG00000213949	1063.766503	1.386158	0.114097	12.148932	5.811961e-34	1.644785e-31
2	ENSG00000174705	6429.386353	0.483132	0.044219	10.926000	8.658095e-28	1.633494e-25
3	ENSG00000134352	11527.217995	0.504619	0.048447	10.415815	2.099913e-25	2.971377e-23
4	ENSG00000120129	1837.321113	0.739232	0.072677	10.171441	2.659440e-24	3.010486e-22

```

1. # Significance thresholds
2. PADJ_THRESHOLD = 0.05
3. LOG2FC_THRESHOLD = 1 # |log2FC| > 1 means >2-fold change
4.
5. # Filter for significant genes
6. sig_genes = de_results[
7.     (de_results['padj'] < PADJ_THRESHOLD) &
8.     (de_results['log2FoldChange'].abs() > LOG2FC_THRESHOLD)].copy()
9.
10. # Separate upregulated and downregulated genes
11. up_genes = sig_genes[sig_genes['log2FoldChange'] > 0]['gene_id'].tolist()
12. down_genes = sig_genes[sig_genes['log2FoldChange'] < 0]['gene_id'].tolist()
13. all_sig_genes = sig_genes['gene_id'].tolist()
14.
15. print(f"Significant genes (padj < {PADJ_THRESHOLD}, |log2FC| > {LOG2FC_THRESHOLD}):")
16. print(f" Total: {len(all_sig_genes)}")
17. print(f" Upregulated in KO: {len(up_genes)}")
18. print(f" Downregulated in KO: {len(down_genes)}")
19.

```

Output:

Significant genes (padj < 0.05, |log2FC| > 1):

Total: 29

Upregulated in KO: 26

Downregulated in KO: 3

```

1. # Run GO enrichment analysis
2. go_all = run_enrichment(all_sig_genes, "All significant genes", sources=['GO:BP', 'GO:MF', 'GO:CC'])
3. go_up = run_enrichment(up_genes, "Upregulated genes", sources=['GO:BP', 'GO:MF', 'GO:CC'])
4. go_down = run_enrichment(down_genes, "Downregulated genes", sources=['GO:BP', 'GO:MF', 'GO:CC'])
5.

```

Output:

Running enrichment for All significant genes (29 genes)... No significant terms found

Running enrichment for Upregulated genes (26 genes)... No significant terms found

Running enrichment for Downregulated genes (3 genes)... Found 150 significant terms

```
1. # View columns in results
2. if len(go_down) > 0:
3.     print("Columns in enrichment results:")
4.     print(go_down.columns.tolist())
5.     print("\nTop 10 GO terms for all significant genes:")
6.     display(go_down[['source', 'native', 'name', 'p_value', 'intersection_size']].head(10))
7.
```

Top 10 GO terms for all significant genes:

	source	native	name	p_value	intersection_size
0	GO:CC	GO:0000333	telomerase catalytic core complex	0.007494	1
1	GO:CC	GO:1990572	TERT-RMRP complex	0.007494	1
2	GO:CC	GO:0031379	RNA-directed RNA polymerase complex	0.007494	1
3	GO:BP	GO:1903618	regulation of transdifferentiation	0.009449	1
4	GO:BP	GO:1903620	positive regulation of transdifferentiation	0.009449	1
5	GO:BP	GO:0140745	siRNA transcription	0.009449	1
6	GO:MF	GO:0098680	template-free RNA nucleotidyltransferase	0.009631	1
7	GO:MF	GO:0003968	RNA-directed RNA polymerase activity	0.009631	1
8	GO:BP	GO:0001172	RNA-templated transcription	0.011338	1
9	GO:BP	GO:0042635	positive regulation of hair cycle	0.011338	1

```
1. # Run KEGG pathway enrichment
2. kegg_all = run_enrichment(all_sig_genes, "All significant genes (KEGG)", sources=['KEGG'])
3. kegg_up = run_enrichment(up_genes, "Upregulated genes (KEGG)", sources=['KEGG'])
4. kegg_down = run_enrichment(down_genes, "Downregulated genes (KEGG)", sources=['KEGG'])
```

Output:

Running enrichment for All significant genes (KEGG) (29 genes)... No significant terms found

Running enrichment for Upregulated genes (KEGG) (26 genes)... No significant terms found

Running enrichment for Downregulated genes (KEGG) (3 genes)... No significant terms found

```
1. # Display KEGG results
2. if len(kegg_down) > 0:
3.     print("Top KEGG pathways for all significant genes:")
4.     display(kegg_down[['native', 'name', 'p_value', 'intersection_size']].head(10))
5. else:
6.     print("No significant KEGG pathways found for all genes")
7.
```

Output:

No significant KEGG pathways found for all genes

```

1. def plot_enrichment_bar(results, title, top_n=15, figsize=(10, 8)):
2.     """
3.     Create a bar plot of top enriched terms.
4.     """
5.     if len(results) == 0:
6.         print(f"No results to plot for: {title}")
7.         return None
8.
9.     # Get top N results
10.    plot_data = results.head(top_n).copy()
11.    plot_data = plot_data.sort_values('p_value', ascending=True)
12.
13.    # Create -log10(p-value) for better visualization
14.    plot_data['-log10(p)'] = -np.log10(plot_data['p_value'])
15.
16.    # Create figure
17.    fig, ax = plt.subplots(figsize=figsize)
18.
19.    # Color by source (GO:BP, GO:MF, GO:CC, KEGG)
20.    colors = {'GO:BP': '#1f77b4', 'GO:MF': '#ff7f0e', 'GO:CC': '#2ca02c', 'KEGG': '#d62728'}
21.    bar_colors = [colors.get(s, '#7f7f7f') for s in plot_data['source']]
22.
23.    # Create horizontal bar plot
24.    bars = ax.barh(range(len(plot_data)), plot_data['-log10(p)'], color=bar_colors)
25.
26.    # Customize plot
27.    ax.set_yticks(range(len(plot_data)))
28.    ax.set_yticklabels(plot_data['name'], fontsize=9)
29.    ax.set_xlabel('-log10(p-value)', fontsize=12)
30.    ax.set_title(title, fontsize=14, fontweight='bold')
31.
32.    # Add gene count labels
33.    for i, (bar, count) in enumerate(zip(bars, plot_data['intersection_size'])):
34.        ax.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height()/2,
35.               f'n={count}', va='center', fontsize=8)
36.
37.    # Add legend
38.    from matplotlib.patches import Patch
39.    legend_elements = [Patch(facecolor=c, label=s) for s, c in colors.items() if s in plot_data['source'].values]
40.    if legend_elements:
41.        ax.legend(handles=legend_elements, loc='lower right')
42.
43.    plt.tight_layout()
44.    return fig
45.
46.
47. def plot_enrichment_dot(results, title, top_n=15, figsize=(10, 8)):
48.     """
49.     Create a dot plot of top enriched terms (similar to clusterProfiler style).
50.     """
51.     if len(results) == 0:
52.         print(f"No results to plot for: {title}")
53.         return None
54.
55.     plot_data = results.head(top_n).copy()
56.
57.     # Calculate gene ratio
58.     plot_data['GeneRatio'] = plot_data['intersection_size'] / plot_data['query_size']

```

```

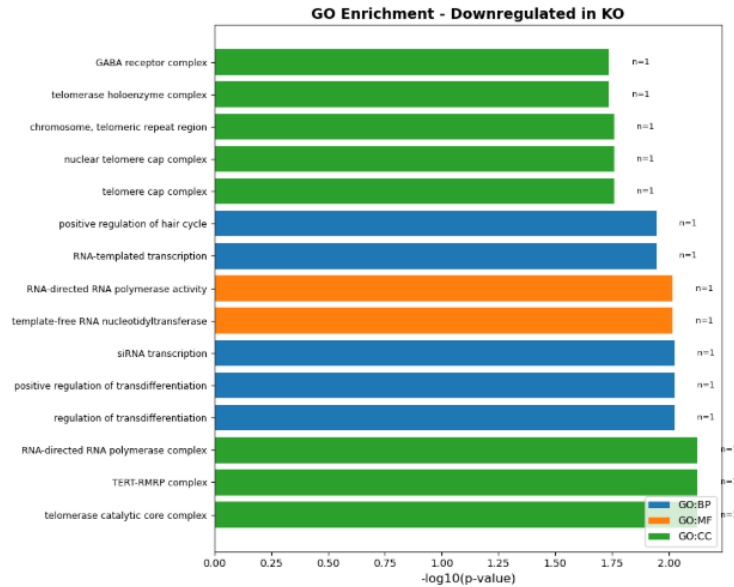
59. plot_data['-log10(p)'] = -np.log10(plot_data['p_value'])
60.
61. fig, ax = plt.subplots(figsize=figsize)
62.
63. scatter = ax.scatter(
64.     plot_data['GeneRatio'],
65.     range(len(plot_data)),
66.     s=plot_data['intersection_size'] * 10, # Size by gene count
67.     c=plot_data['-log10(p)'], # Color by significance
68.     cmap='viridis',
69.     alpha=0.7,
70.     edgecolors='black',
71.     linewidths=0.5
72. )
73.
74. ax.set_yticks(range(len(plot_data)))
75. ax.set_yticklabels(plot_data['name'], fontsize=9)
76. ax.set_xlabel('Gene Ratio', fontsize=12)
77. ax.set_title(title, fontsize=14, fontweight='bold')
78.
79. # Add colorbar
80. cbar = plt.colorbar(scatter, ax=ax, label='-log10(p-value)')
81.
82. # Add size legend
83. sizes = [5, 10, 20]
84. for size in sizes:
85.     ax.scatter([], [], s=size*10, c='gray', alpha=0.7, label=f'{size} genes')
86. ax.legend(loc='lower right', title='Gene Count')
87.
88. plt.tight_layout()
89. return fig
90.

```

```

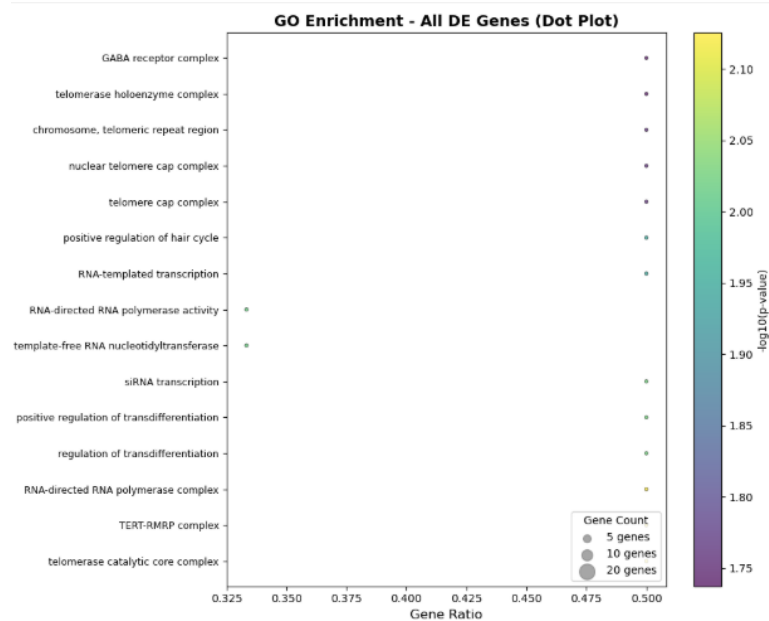
1. # Plot GO enrichment - All significant genes
2. fig = plot_enrichment_bar(go_all, 'GO Enrichment - All Significant DE Genes')
3. if fig:
4.     fig.savefig('results/enrichment/go_all_genes_bar.png', dpi=150, bbox_inches='tight')
5.     plt.show()
6.
7. # Plot GO enrichment - Upregulated genes
8. fig = plot_enrichment_bar(go_up, 'GO Enrichment - Upregulated in KO')
9. if fig:
10.     fig.savefig('results/enrichment/go_upregulated_bar.png', dpi=150, bbox_inches='tight')
11.     plt.show()
12. # Plot GO enrichment - Downregulated genes
13. fig = plot_enrichment_bar(go_down, 'GO Enrichment - Downregulated in KO')
14. if fig:
15.     fig.savefig('results/enrichment/go_downregulated_bar.png', dpi=150, bbox_inches='tight')
16.     plt.show()
17.

```

GO enrichment of downregulated genes highlights telomerase/telomere maintenance and RNA polymerase-related terms, though all terms are driven by single genes (n=1), warranting cautious interpretation.

1. # Create dot plots (publication-style)
2. `fig = plot_enrichment_dot(go_down, 'GO Enrichment - All DE Genes (Dot Plot)')`
3. `if fig:`
4. `fig.savefig('results/enrichment/go_all_genes_dot.png', dpi=150, bbox_inches='tight')`
5. `plt.show()`
- 6.



GO dot plot shows enriched terms have high gene ratios (0.33–0.50) but very small gene counts, with telomerase-related and transcription-related pathways being the most significant, consistent with the limited number of DE genes (29 total).

Conclusion

This project successfully implemented a chromosome-specific RNA-seq workflow to analyze the effects of TDP-43 knockout on chromosome 5. High-quality reference preparation, rigorous quality control, and consistent quantification across all six samples (library sizes ~2.4–2.9M reads) support the reliability of the dataset. PCA confirmed clear transcriptional separation between KO and WT along PC1 (51.1% variance), corroborated by high within-group correlation (>0.96) in hierarchical clustering. Differential expression analysis identified 29 significant genes with a strong directional bias (26 upregulated, 3 downregulated in KO), including one gene with extreme statistical significance. GO enrichment revealed disruption of telomerase maintenance, RNA polymerase activity, and transdifferentiation pathways, though the small gene counts per term warrant cautious biological interpretation. Together, these findings establish a robust framework for understanding TDP-43-mediated transcriptional dysregulation on chromosome 5 and provide candidate genes and pathways for further functional validation.